Abstract Method Exercise

A vehicle rental company rents different types of vehicles, such as cars and bicycles. Every vehicle has a brand and can display its information. Different vehicles calculate rental prices differently. Some vehicles can also be started and stopped. Develop a simple rental system using an abstract class, an interface, and concrete classes.

1. Create an abstract class Vehicle

- Declare a public property Brand of type string.
- Create a constructor Vehicle(string brand) that initializes the property.
- Declare an abstract method CalculateRentalPrice(int days) returning double.
- Implement a concrete method ShowInfo() that prints the vehicle's brand.

2. Create an interface IStartable

- Declare void Start() and void Stop().

3. Create a concrete class Car

- Inherit from Vehicle and implement IStartable.
- Create a constructor that receives the brand and passes it to the base class.
- Override CalculateRentalPrice(int days): 300 DKK per day.
- Implement Start() to print "Car started" and Stop() to print "Car stopped".

4. Create a concrete class Bicycle

- Inherit from Vehicle.
- Create a constructor that receives the brand and passes it to the base class.
- Override CalculateRentalPrice(int days): 100 DKK per day.
- Do not implement IStartable.

5. Test the classes in Main()

- Create a Car named "Toyota" and a Bicycle named "Trek".
- Store both objects in a List<Vehicle>.
- Call ShowInfo() and CalculateRentalPrice(3) separately on each object and print the results.
- Create an IStartable variable referencing the car; call Start() and Stop().

Starter Structure

```
public abstract class Vehicle
{
    // TODO: Property, constructor,
    // abstract method, concrete method
}

// TODO: Create IStartable interface
// TODO: Create Car class
// TODO: Create Bicycle class

public class Program
{
    public static void Main(string[] args)
    {
        // TODO: Create Car and Bicycle objects
        // TODO: Call ShowInfo() on both objects
        // TODO: Print each object's rental price for 3 days
        // TODO: Test IStartable with the Car object
    }
}
```

Expected Output

```
Brand: Toyota
Rental price for 3 days: 900 DKK
Brand: Trek
Rental price for 3 days: 300 DKK
Car started
Car stopped
```

Think About

1. Why is Vehicle abstract rather than concrete?
2. Why must Car and Bicycle override CalculateRentalPrice()?
3. Why can both objects be stored in List<Vehicle>?
4. Why does Car implement IStartable, but Bicycle does not?
5. What is the difference between inheriting from Vehicle and implementing IStartable?